



DESARROLLO BIG DATA CON PYTHON

TEMA 3. Arquitectura Spark y
Fundamentos de PySpark



Contenidos Tema 3

- / 1. Visión general y Arquitectura de Apache Spark
- / 2. Inicialización de Spark en Python: *SparkSession*
- / 3. Concepto de RDDs: Datasets Distribuidos
- / 4. DataFrames en PySpark
- / 5. LAB: Configuración PySpark en Google Colab y carga de datasets

1. Visión general y arquitectura de Apache Spark

Los límites del Python clásico frente al Big Data

- Librerías tradicionales para el manejo de datasets, como **Pandas**, requieren la carga de todo el dataset en la memoria RAM → riesgo de **MemoryError** con datasets grandes
- **Escalabilidad en vertical** (*scaling up*): adquirir más RAM y CPU es caro y existen limitaciones físicas.
- **Escalabilidad horizontal** (*scaling out*): conectar varias máquinas más sencillas para que trabajen en equipo, como un *cluster*. Barato y muy escalable.

Spark, y más concretamente PySpark, apuesta por la escalabilidad horizontal.

Apache Spark: es un motor unificado muy eficiente para procesar datos a gran escala.

- Computación en memoria, cientos de veces más rápido que paradigmas como *Hadoop MapReduce*.

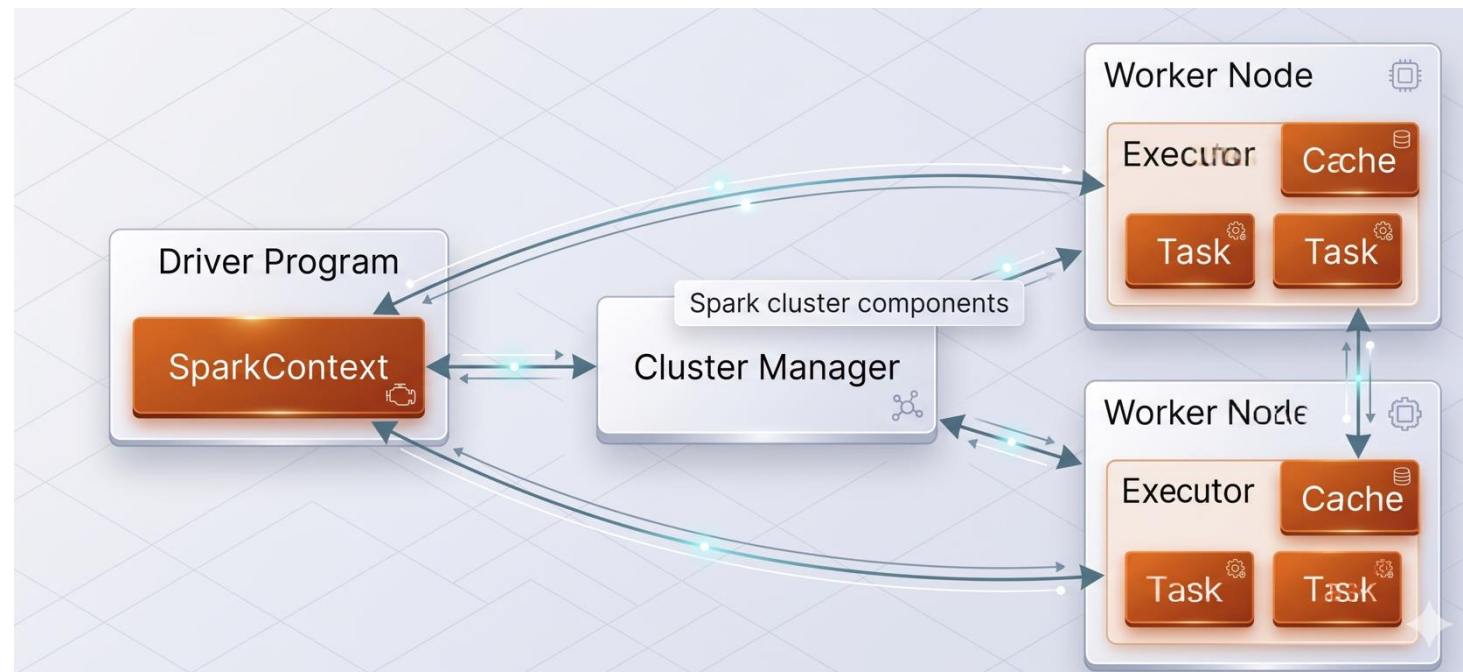
PySpark: API de Python para utilizar Spark. Internamente el código Python se traduce y ejecuta en el lenguaje *Scala*, que es el lenguaje nativo en el que está escrito Spark.



1. Visión general y arquitectura de Apache Spark

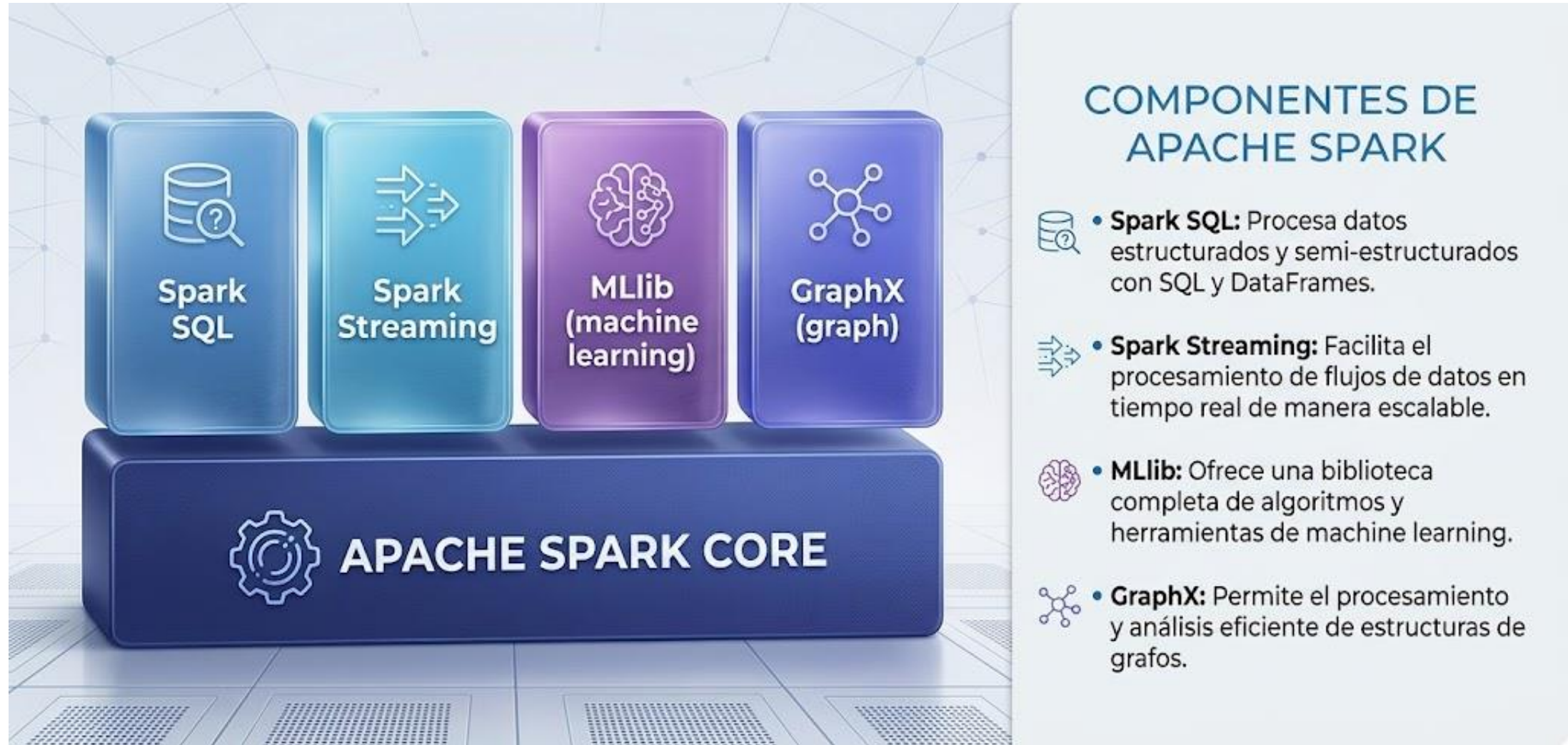
Arquitectura de Spark

- **Driver Program:** es el nodo maestro, donde tenemos nuestro programa Python (IDE o session de Colab). Desde aquí se organiza y reparte el trabajo de procesamiento.
- **Cluster Manager:** gestor de recursos, asigna memoria y CPU
- **Worker nodes / Executors:** nodos "obreros". Cada uno cuenta con su propia memoria, y ejecuta las tareas que les da el nodo maestro (Driver), almacenando resultados en particiones de datos.



1. Visión general y arquitectura de Apache Spark

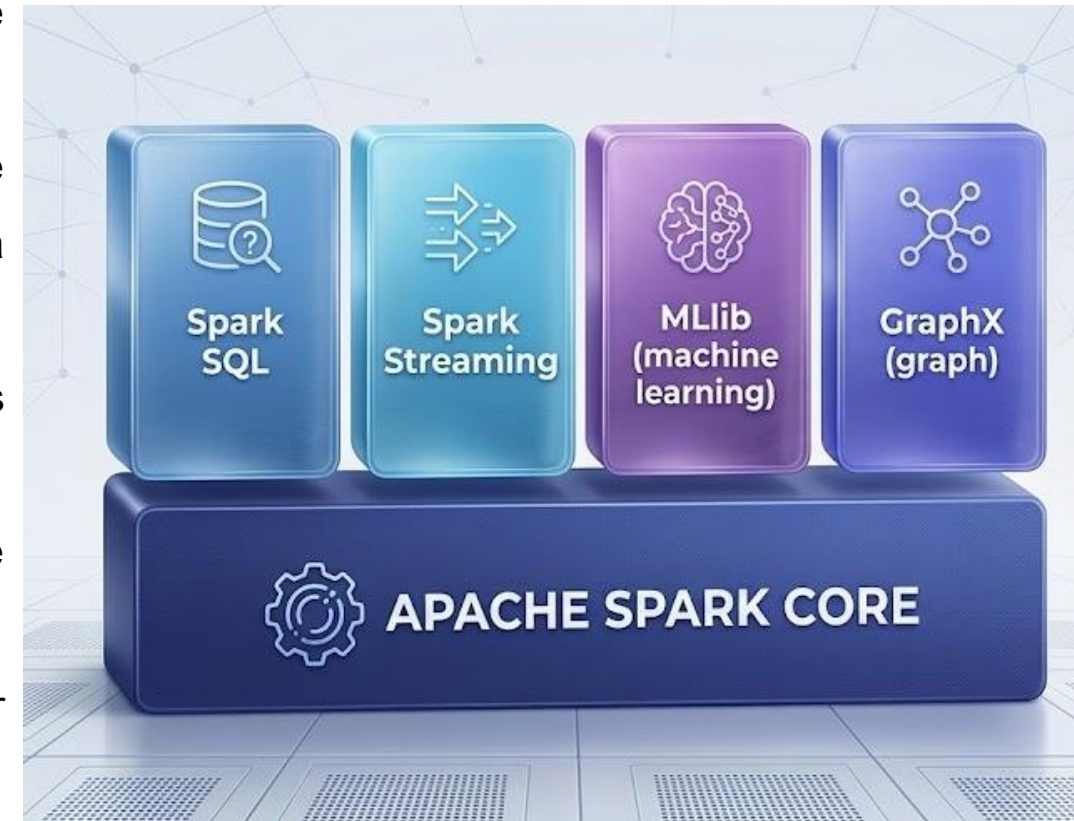
Principales componentes de Spark



1. Visión general y arquitectura de Apache Spark

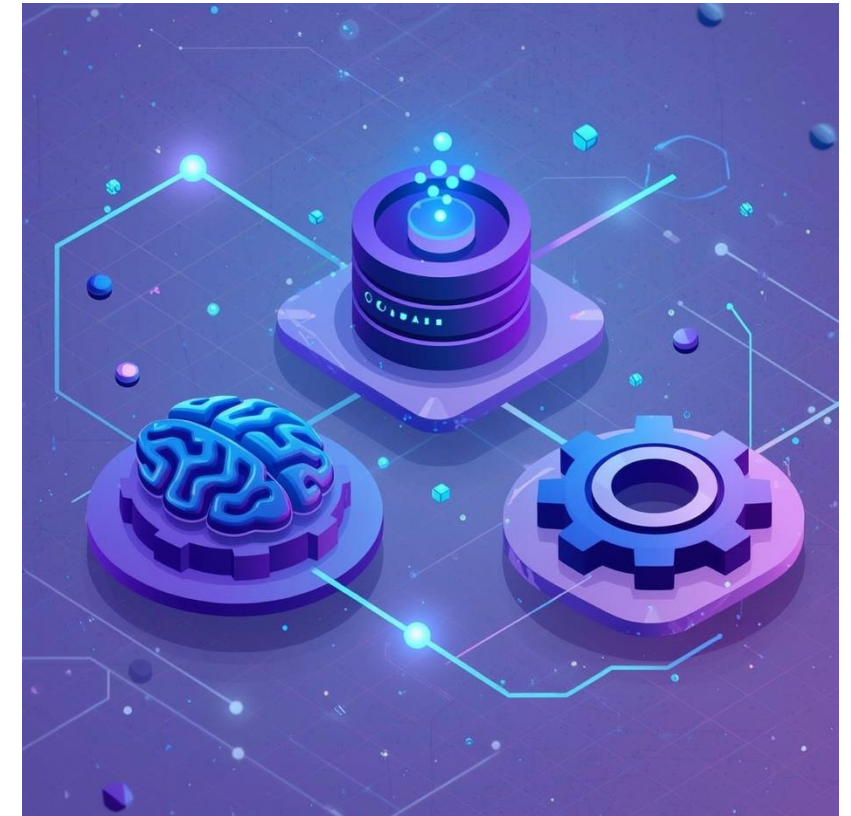
Principales componentes de Spark

- **Spark SQL:** Permite el procesamiento de datos estructurados y semiestructurados mediante el uso de consultas SQL y DataFrames.
- **Spark Streaming:** Habilita el procesamiento de flujos de datos en tiempo real de manera escalable y tolerante a fallos.
- **MLlib:** Biblioteca completa de algoritmos y herramientas eficientes para aprendizaje automático (machine learning).
- **GraphX:** Facilita el procesamiento y el análisis unificado de estructuras de grafos y datos tabulares.
- **Apache Spark Core:** Motor central responsable de ejecutar tareas, gestión de memoria y recuperación de fallos.



El punto de entrada unificado para interactuar con Spark en Python

- **¿Qué es SparkSession?** Es la "puerta de enlace" única de una aplicación basada en Spark. Objeto necesario para leer datos, crear DataFrames distribuidos y ejecutar SQL.
- Se instancia usando `SparkSession.builder`, lo que permite configurar las propiedades de la app de forma encadenada.
- **Patrón singleton:** mediante `.getOrCreate()`, aseguramos que solo exista una sesión activa en nuestra aplicación. Si ya hubiese una en memoria, se reutiliza.



2. Inicialización de Spark en Python: SparkSession

1. Instalación y Setup

Instalar PySpark (!pip install pyspark en Colab o vs Code) e importar la clase SparkSession.

2. Construcción de la Sesión

Definir el nombre y el master ("local[*]" para usar todos los núcleos locales).

3. Implementar la Lógica de tu programa

Usar la sesión instanciada (spark) para crear un *DataFrame* a partir de una lista de Python y mostrarlo.

4. Cerrar/detener la Sesión

Ejecutar `spark.stop()` al final para liberar los recursos del clúster.

```
# !pip install pyspark

from pyspark.sql import SparkSession

# 1. Inicializar SparkSession
spark = SparkSession.builder \
    .appName("MiPrimeraApp") \
    .master("local[*]") \
    .getOrCreate()

# 2. Crear datos ficticios
data = [("Alice", 25), ("Bob", 30)]
cols = ["Nombre", "Edad"]

# 3. Crear y mostrar DataFrame
df = spark.createDataFrame(data, cols)
df.show()

# 4. Detener la sesión
spark.stop()
```

Más ejemplos prácticos de configuración de **SparkSession** en la libreta de código T3_1

3. Concepto de RDDs: Datasets Distribuidos

Resilient Distributed Dataset (RDD)

- Es la colección de datos fundamental y primitiva sobre la que opera PySpark (y Spark al completo).
- **Resiliente:** tolerante a fallos. Si un ordenador del cluster en el que se reparten los datos falla, el sistema reconstruirá el fragmento perdido usando el histórico de operaciones.
- **Distribuido:** el dataset es troceado en "particiones" procesadas en paralelo en múltiples máquinas.
- **Inmutable:** una vez creado, un RDD no se puede modificar. Toda modificación se guarda en un objeto RDD nuevo, para evitar problemas de concurrencia en el procesamiento paralelo.



3. Concepto de RDDs: Datasets Distribuidos

Limitaciones de los RDDs

- **Ceguera de optimización:** Spark trata los RDDs como cajas negras opacas, no aplicando optimizaciones automáticas para agilizar la ruta de ejecución (existen optimizadores como *Catalyst* para hacer frente a esto).
- **Lentitud de serialización (en PySpark):** cuando el código sobre RDDs está escrito en Python, hay que traducir, empaquetar y enviar datos constantemente entre entornos de memoria, lo que merma el rendimiento.
- **Falta de esquema:** los RDDs nativos no tienen formato tabular con columnas nombradas y tipos de datos asociados. Esto obliga a escribir código manual más complejo y proclive a errores para realizar operaciones de análisis sencillas.
- **Saturación de memoria:** la inmutabilidad y generación de objetos nuevos en memoria tras cada transformación puede saturar los procesos de limpieza de memoria fácilmente.



El origen de los DataFrames → Pandas

La librería más utilizada para manipulación y tratamiento de datos estructurados tabularmente.

- `pd.read_csv()` crea un DataFrame -la principal estructura de Pandas para datos estructurados o tabulares- a partir de un fichero.
- Atributos básicos del DataFrame como *shape*, *columns*, permiten conocer las dimensiones y columnas del dataset.

Muestra los primeros cinco
elementos del DataFrame

```
import pandas as pd

# Cargar el dataset
df = pd.read_csv("50_Startups.csv")

# Exploración básica
print(df.shape)
print(df.columns)
df.head()
print(df.describe())

# Acceso a una Series (estructura 1D)
gastos_marketing = df["Marketing Spend"]
print(gastos_marketing.head())

# Guardar el DataFrame
df.to_csv("50_Startups_copia.csv", index=False)
```

Muestra un resumen
estadístico sencillo del dataset

Breve repaso de Pandas

Selección de datos en un DataFrame de Pandas

- Selección de una columna (atributo del dataset):

```
df = pd.read_csv("50_Startups.csv")
```

```
marketing = df["Marketing Spend"]
```

Se guarda el atributo como un objeto Series

```
marketing_df = df[["Marketing Spend"]]
```

Se guarda el atributo como un DataFrame (bidimensional) de una sola columna

- Selección de una fila (instancia u observación del dataset):

```
print(df.iloc[0])
```

```
print(df.iloc[[0]])
```

Devuelve un Series cuyos índices son los nombres de las columnas del DataFrame *df*

Devuelve un DataFrame de una sola fila

```
type(marketing)  
type(marketing_df)
```

```
<class 'pandas.core.series.Series'>  
<class 'pandas.core.frame.DataFrame'>
```

Breve repaso de Pandas- resumen de selección de datos

Tipo de acceso	Sintaxis	Ejemplo	Resultado
Columna de un <i>df</i> (una)	<code>df["col"]</code>	<code>df["A"]</code>	<i>Series</i> con la columna "A"
Columnas de un <i>df</i> (varias)	<code>df[["A", "B"]]</code>	<code>df[["A", "B"]]</code>	<i>DataFrame</i> con columnas A y B
Fila por posición	<code>df.iloc[i]</code>	<code>df.iloc[0]</code>	<i>Series</i> con la fila 0
Fila por etiqueta	<code>df.loc["f1"]</code>	<code>df.loc["f1"]</code>	<i>Series</i> con la fila "f1"
Filas (varias)	<code>df.iloc[[i, j]]</code>	<code>df.iloc[[0, 2]]</code>	<i>DataFrame</i> con filas 0 y 2
Celda [fila, col]	<code>df.loc[i, "col"]</code> <code>df.iloc[i, j]</code>	<code>df.loc["f1", "A"]</code> <code>df.iloc[0, 1]</code>	Valor escalar

Tipo de acceso	Sintaxis	Ejemplo o caso equivalente	Resultado
Slicing de filas	<code>df[a:b]</code>	<code>df[1:3]</code>	Filas 1 y 2
Slicing por posición	<code>df.iloc[a:b, :]</code>	<code>df.iloc[1:3, :]</code>	Filas 1 y 2
Slicing por etiquetas	<code>df.loc["f1":"f3"]</code>	<code>df.loc["f1":"f2"]</code>	Rango inclusivo por índices
Filtro booleano (<i>df</i>)	<code>df[condición]</code>	<code>df[df["A"] > 15]</code>	Filas donde A > 15
Filtro booleano (<i>Series</i>)	<code>serie[condición]</code>	<code>s[s > 0]</code>	Valores mayores a 0

Aunque no es habitual, puede hacerse *slicing* para seleccionar un rango de columnas consecutivas, debiendo hacerse así:

```
df.iloc[:, 1:3] # Todas las filas, columnas 1 y 2 (no incluye la 3)
```

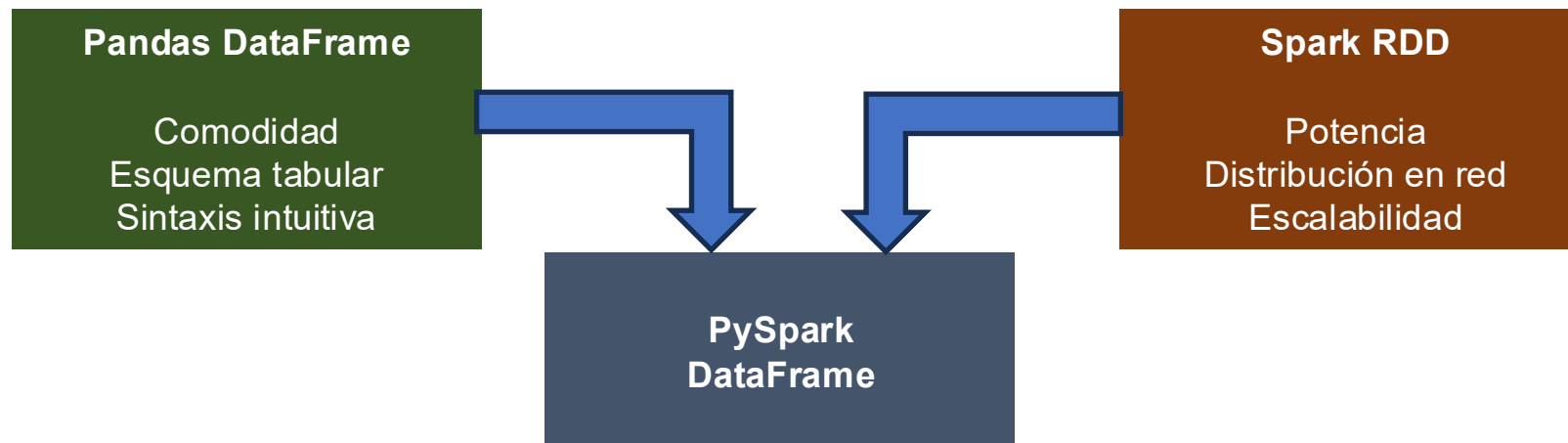
4. DataFrames en PySpark

La gran limitación de Pandas

- Pandas actúa confinado en la RAM de una sola máquina: no está diseñado para procesamiento distribuido.
- Perfecto para *datasets* pequeños-medianos, pero propenso a un `MemoryError` para *datasets* mayores.

La SOLUCIÓN: El DataFrame de PySpark

Aúna las ventajas de los RDDs y los Dataframes de Pandas.



4. DataFrames en PySpark

- **Pandas DataFrame:** único bloque de datos, reside en un único núcleo y bajo una sola memoria RAM.
- **PySpark DataFrame:** dataset distribuido (por debajo actúa como RDD, *Resilient Distributed Dataset* de Spark). Aunque a nivel lógico parezca un solo dataset, físicamente está particionado y distribuido por todo el cluster de máquinas.
- La sintaxis se mantiene similar a Pandas, pese a que el motor subyacente es muy diferente.

```
print("\nCargando datos con PySpark SQL...")

df = spark.read.csv(
    "paysim_sample.csv",
    header=True, # La primera fila tiene los nombres de las columnas
    inferSchema=True # PySpark adivina el tipo de dato (entero, string, float)
)

print("Esquema inferido por Spark:")
df.printSchema()

print("Primeras 5 filas distribuidas:")
df.show(5)
```

Evaluación perezosa con PySpark

- Spark no ejecuta las instrucciones de filtrado, selección, etc. Cuando se llega a la instrucción que las contiene.
- Se crea un plan de ejecución con la forma de un grafo acíclico dirigido (DAG).
- ¿Cuándo se ejecuta todo? Cuando se llega a ciertas acciones de ejecución inmediata (*eager*, evaluación inmediata) como **show**, **count**, **collect**.

```
df_filtered = df.filter(df.amount > 5000) \
    .select("type", "amount", "oldbalanceOrg", "isFraud")

# En este punto, no se ha ejecutado nada. Solo se ha definido el
# plan mediante un DAG interno.
print("Conteo de transacciones grandes:", df_filtered.count()) #
# Aquí se dispara la acción
```

5. Lab: Configuración PySpark en Google Colab y Carga de Datasets

En la libreta Python que crearemos...

- Crearemos un dataset sintético pero realista (para evitar descargas muy pesadas) que emula un dataset de gran tamaño llamado **PaySim** → **transacciones financieras**
- Realizaremos la ingesta, limpieza y *feature engineering* usando un módulo PySpark SQL.

EJERCICIO PROPUESTO: carga y procesamiento con PySpark del dataset de viviendas de California, disponible en: <https://github.com/gakudo-ai/open-datasets>

